

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 3.4 — Deploying and Implementing Data Solutions

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 3: Deploying and implementing a cloud solution (~25 % of the exam)**. You must know how to deploy data products (Cloud SQL, Firestore, BigQuery, Spanner, Pub/Sub, Dataflow, Cloud Storage, AlloyDB) and load data from various sources.

1. Deploying Cloud SQL

Creating a Cloud SQL Instance

```
# Create a MySQL instance
gcloud sql instances create my-mysql \
  --database-version=MYSQL_8_0 \
  --tier=db-n1-standard-4 \
  --region=us-central1 \
  --root-password=SECURE_PASSWORD \
  --storage-type=SSD \
  --storage-size=100GB \
  --storage-auto-increase \
  --backup-start-time=02:00 \
  --enable-bin-log \
  --availability-type=REGIONAL

# Create a PostgreSQL instance
gcloud sql instances create my-postgres \
  --database-version=POSTGRES_15 \
  --tier=db-custom-4-16384 \
  --region=us-central1 \
  --root-password=SECURE_PASSWORD \
  --availability-type=REGIONAL

# Create a SQL Server instance
gcloud sql instances create my-sqlserver \
  --database-version=SQLSERVER_2019_STANDARD \
  --tier=db-custom-4-16384 \
  --region=us-central1 \
  --root-password=SECURE_PASSWORD
```

Key Configuration Options

Option	Description
<code>--availability-type=REGIONAL</code>	High availability (failover replica in another zone)
<code>--availability-type=ZONAL</code>	Single zone (no failover)
<code>--storage-auto-increase</code>	Automatically grow storage as needed
<code>--enable-bin-log</code>	Required for point-in-time recovery (MySQL)
<code>--backup-start-time</code>	Daily backup window (UTC)
<code>--maintenance-window-day</code>	Preferred maintenance day
<code>--database-flags</code>	Set database configuration flags
<code>--authorized-networks</code>	Allow access from specific IPs

Connecting to Cloud SQL

```
# Connect using Cloud SQL Auth Proxy (recommended)
# Download the proxy
curl -o cloud-sql-proxy https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy
chmod +x cloud-sql-proxy

# Start the proxy
./cloud-sql-proxy PROJECT_ID:REGION:INSTANCE_NAME

# Connect via gcloud (interactive)
gcloud sql connect my-mysql --user=root

# Configure private IP connectivity
gcloud sql instances patch my-mysql \
  --network=projects/PROJECT_ID/global/networks/my-vpc \
  --no-assign-ip # Remove public IP
```

Creating Databases and Users

```
# Create a database
gcloud sql databases create my-database --instance=my-mysql

# Create a user
gcloud sql users create my-user \
  --instance=my-mysql \
  --password=USER_PASSWORD

# List databases
gcloud sql databases list --instance=my-mysql

# List users
gcloud sql users list --instance=my-mysql
```

2. Deploying Firestore

Setting Up Firestore

```
# Create a Firestore database (Native mode — default for new projects)
gcloud firestore databases create \
  --location=us-central \
  --type=firestore-native

# Create a Firestore database in Datastore mode
gcloud firestore databases create \
  --location=us-central \
  --type=datastore-mode
```

Firestore Mode Selection

Mode	Use Case	Features
Native	Mobile/web apps, real-time sync	Real-time listeners, offline support, subcollections
Datastore	Server-side applications	Compatible with legacy Datastore, entity groups

Important: Mode **cannot be changed** after database creation. Choose carefully.

Firestore Data Model

```
Database
└─ Collection: "users"
    │   └─ Document: "user123"
    │       │   └─ name: "Alice"
    │       │   └─ email: "alice@example.com"
    │       │       └─ Subcollection: "orders"
    │       │           └─ Document: "order001"
    │       │           └─ Document: "order002"
    └─ Document: "user456"
        │   └─ name: "Bob"
        └─ email: "bob@example.com"
```

Firestore Indexes

- **Single-field indexes** — Created automatically for each field
- **Composite indexes** — Must be created manually for complex queries
- Indexes are required for all Firestore queries

```
# Create a composite index
gcloud firestore indexes composite create \
  --collection-group=users \
  --field-config=field-path=name,order=ASCENDING \
  --field-config=field-path=created,order=DESCENDING

# List indexes
gcloud firestore indexes composite list
```

3. Deploying BigQuery

Creating Datasets and Tables

```
# Create a dataset
bq mk --dataset \
  --location=US \
  --description="Sales data" \
  PROJECT_ID:sales_data

# Create a table with schema
bq mk --table \
  PROJECT_ID:sales_data.transactions \
  order_id:STRING, customer_id:STRING, amount:FLOAT, timestamp:TIMESTAMP

# Create a table from a schema file
bq mk --table \
  PROJECT_ID:sales_data.transactions \
  schema.json

# List datasets
bq ls --project_id=PROJECT_ID

# List tables in a dataset
bq ls PROJECT_ID:sales_data

# Describe a table
bq show PROJECT_ID:sales_data.transactions
```

BigQuery Table Types

Type	Description	Use Case
Native table	Data stored in BigQuery columnar storage	Default, best performance
External table	Schema in BQ, data in Cloud Storage/Bigtable/Drive	Query data without loading
View	Virtual table defined by SQL query	Simplify complex queries
Materialized view	Precomputed query results, auto-updated	Improve query performance
Partitioned table	Table divided by time or integer range	Reduce query cost and improve performance
Clustered table	Data sorted by specified columns	Further reduce scan costs

Partitioning and Clustering

```
-- Create a partitioned and clustered table
CREATE TABLE `project.dataset.events`
(
  event_id STRING,
  event_type STRING,
  user_id STRING,
  event_timestamp TIMESTAMP,
  region STRING
)
PARTITION BY DATE(event_timestamp)
CLUSTER BY region, event_type;
```

- **Partitioning:** Divides table into segments (by date, timestamp, integer range, or ingestion time)
- **Clustering:** Sorts data within partitions by specified columns
- Both reduce the amount of data scanned → lower cost

4. Deploying Cloud Spanner

```
# Create a Spanner instance
gcloud spanner instances create my-spanner-instance \
  --config=regional-us-central1 \
  --description="Production Spanner" \
  --processing-units=1000

# Create a database
gcloud spanner databases create my-database \
  --instance=my-spanner-instance

# Execute DDL to create tables
gcloud spanner databases ddl update my-database \
  --instance=my-spanner-instance \
  --ddl='CREATE TABLE Users (
    UserId INT64 NOT NULL,
    Name STRING(100),
    Email STRING(200),
    Created TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true)
  ) PRIMARY KEY (UserId)'
```

Spanner Instance Configurations

Config Type	Example	Description
Regional	<code>regional-us-central1</code>	Single region, 99.999% SLA
Multi-region	<code>nam6</code> (North America)	Multiple regions, highest availability
Dual-region	<code>nam-eur-asia1</code>	Specific dual-region config

Processing Units vs. Nodes

- **1 node = 1000 processing units**
- Minimum: 100 processing units (for development)
- Processing units can be scaled incrementally

5. Deploying Pub/Sub

Creating Topics and Subscriptions

```
# Create a topic
gcloud pubsub topics create my-topic

# Create a pull subscription
gcloud pubsub subscriptions create my-pull-sub \
  --topic=my-topic \
  --ack-deadline=60 \
  --message-retention-duration=7d

# Create a push subscription
gcloud pubsub subscriptions create my-push-sub \
  --topic=my-topic \
  --push-endpoint=https://my-service.run.app/push \
  --ack-deadline=30

# Create a BigQuery subscription (direct write to BQ)
gcloud pubsub subscriptions create my-bq-sub \
  --topic=my-topic \
  --bigquery-table=PROJECT_ID:dataset.table

# Create a Cloud Storage subscription
gcloud pubsub subscriptions create my-gcs-sub \
  --topic=my-topic \
  --cloud-storage-bucket=my-bucket \
  --cloud-storage-file-prefix=pubsub/ \
  --cloud-storage-file-suffix=.json
```

Publishing Messages

```
# Publish a message
gcloud pubsub topics publish my-topic --message="Hello, World!"

# Publish with attributes
gcloud pubsub topics publish my-topic \
  --message="Order created" \
  --attribute="type=order,priority=high"

# Pull messages (for testing)
gcloud pubsub subscriptions pull my-pull-sub --auto-ack --limit=10
```

Key Pub/Sub Concepts

Concept	Description
Topic	Named channel for sending messages
Subscription	Named resource for receiving messages from a topic
Pull	Subscriber requests messages from the service
Push	Service delivers messages to an HTTP endpoint
Acknowledgement	Subscriber confirms message processing
Dead letter topic	Receives messages that failed processing after max retries
Message retention	How long unacknowledged messages are kept (default: 7 days)
Ordering keys	Guarantee message ordering for messages with the same key
Filtering	Subscription filters to receive only matching messages

6. Deploying Dataflow

What Is Dataflow?

- Fully managed service for **stream and batch data processing**
- Based on **Apache Beam** programming model
- Supports Java, Python, and Go SDKs
- Autoscaling workers, exactly-once processing

Running a Dataflow Job

```
# Run a batch job from a template (e.g., Cloud Storage to BigQuery)
gcloud dataflow jobs run my-batch-job \
  --gcs-location=gs://dataflow-templates-us-central1/latest/GCS_Text_to_BigQuery \
  --region=us-central1 \
  --parameters=\
inputFilePattern=gs://my-bucket/data/*.csv,\
JSONPath=gs://my-bucket/schema.json,\
outputTable=PROJECT_ID:dataset.table,\
bigQueryLoadingTemporaryDirectory=gs://my-bucket/temp/

# Run a streaming job (Pub/Sub to BigQuery)
gcloud dataflow jobs run my-streaming-job \
  --gcs-location=gs://dataflow-templates-us-central1/latest/PubSub_to_BigQuery \
  --region=us-central1 \
  --parameters=\
inputTopic=projects/PROJECT_ID/topics/my-topic,\
outputTableSpec=PROJECT_ID:dataset.table

# List Dataflow jobs
gcloud dataflow jobs list --region=us-central1

# Describe a job
gcloud dataflow jobs describe JOB_ID --region=us-central1

# Cancel a job
gcloud dataflow jobs cancel JOB_ID --region=us-central1

# Drain a streaming job (process remaining data, then stop)
gcloud dataflow jobs drain JOB_ID --region=us-central1
```

Dataflow Templates

Google provides pre-built templates for common patterns: - **Cloud Storage to BigQuery** (CSV, JSON, Avro) - **Pub/Sub to BigQuery** (streaming) - **Pub/Sub to Cloud Storage** (streaming) - **BigQuery to Cloud Storage** (export) - **Cloud Storage to Spanner** - **Jdbc to BigQuery**

7. Deploying Cloud Storage

```
# Create a bucket
gcloud storage buckets create gs://my-unique-bucket \
  --location=us-central1 \
  --default-storage-class=STANDARD \
  --uniform-bucket-level-access

# Create a multi-region bucket
gcloud storage buckets create gs://my-global-bucket \
  --location=US \
  --default-storage-class=STANDARD

# Bucket naming rules:
# - Globally unique
# - 3-63 characters
# - Lowercase letters, numbers, hyphens, underscores, dots
# - Must start and end with letter or number
# - Cannot be an IP address
```

Bucket Configuration

```
# Enable versioning
gcloud storage buckets update gs://my-bucket --versioning

# Set lifecycle rules
gcloud storage buckets update gs://my-bucket \
  --lifecycle-file=lifecycle.json

# Enable uniform bucket-level access
gcloud storage buckets update gs://my-bucket \
  --uniform-bucket-level-access

# Set a retention policy (minimum retention period)
gcloud storage buckets update gs://my-bucket \
  --retention-period=365d

# Set default storage class
gcloud storage buckets update gs://my-bucket \
  --default-storage-class=NEARLINE
```

8. Deploying AlloyDB

```
# Create an AlloyDB cluster
gcloud alloydb clusters create my-cluster \
  --region=us-central1 \
  --password=SECURE_PASSWORD \
  --network=my-vpc

# Create a primary instance
gcloud alloydb instances create my-primary \
  --cluster=my-cluster \
  --region=us-central1 \
  --instance-type=PRIMARY \
  --cpu-count=4

# Create a read pool instance
gcloud alloydb instances create my-read-pool \
  --cluster=my-cluster \
  --region=us-central1 \
  --instance-type=READ_POOL \
  --cpu-count=2 \
  --read-pool-node-count=2
```

9. Bigtable

What Is Bigtable?

- Fully managed, **wide-column NoSQL** database with an **HBase-compatible API**
- Designed for massive scale: petabytes of data with **single-digit millisecond latency**

Use Cases

- Time-series data (IoT telemetry, monitoring metrics)
- Financial market data and transactional logs
- Personalization and recommendation engines
- Any structured dataset **>1 TB** requiring low-latency reads/writes

Instance Types

Type	Description	SLA
PRODUCTION	Multi-node cluster; supports replication	Yes
DEVELOPMENT	Single-node; no replication; cheaper	No

gcloud / cbt Commands

```
gcloud bigtable instances create MY_INSTANCE --display-name="My Instance" --cluster=MY_CLUSTER
gcloud bigtable instances list
gcloud bigtable clusters resize MY_CLUSTER --instance=MY_INSTANCE --num-nodes=5
# Create table using cbt CLI
cbt -instance=MY_INSTANCE createtable MY_TABLE
cbt -instance=MY_INSTANCE createfamily MY_TABLE cfl
cbt -instance=MY_INSTANCE set MY_TABLE row1 cfl:col1=value
cbt -instance=MY_INSTANCE read MY_TABLE
```

Key Design Concepts

- **Row key is the only index** — all reads are by row key or row key range; no secondary indexes
- Design row keys to **distribute load evenly**: avoid monotonic timestamps as a prefix (causes hotspotting); use a reversed timestamp or a hash prefix instead
- **Column families** group related columns; each family has its own garbage collection (GC) policy
- **Replication**: multi-cluster replication provides HA and disaster recovery; consistency between clusters is **eventual**

Exam tip: Bigtable scales **compute** by adding nodes (3-node minimum for production SLA).

Storage is separate from compute and scales independently. More nodes = more throughput, not more storage.

10. Memorystore

Fully managed **Redis** or **Memcached** in GCP — no SSH access, no OS patching, no cluster management.

Use Cases

- Application caching (reduce database load)
- Session management

- Leaderboards and counters
- Pub/Sub message queuing (Redis Streams)

Redis

```
gcloud redis instances create MY_CACHE --size=1 --region=us-central1 --tier=STANDARD_HA --  
gcloud redis instances list --region=us-central1  
gcloud redis instances describe MY_CACHE --region=us-central1  
gcloud redis instances delete MY_CACHE --region=us-central1
```

Tier	Description
BASIC	Single node; no replication; lower cost; no HA
STANDARD_HA	Primary + replica; automatic failover; production-grade

- Access is via **private IP only** — no public endpoint
- The application and the Memorystore instance must be in the **same VPC** (or a connected VPC via peering)

Memcached

```
gcloud memcache instances create MY_MEMCACHE --node-count=3 --node-cpu=1 --node-memory=1GB
```

- **Stateless** — no persistence, no replication; data is lost on node restart
- Supports **multi-threading** for higher throughput on CPU-bound caching workloads
- Use when you need simple key-value caching and multi-threading; use Redis when you need persistence, data structures, or HA

9. Loading Data

Command Line Upload (gsutil / gcloud storage)

```
# Upload a single file to Cloud Storage
gcloud storage cp local-file.csv gs://my-bucket/data/

# Upload a directory recursively
gcloud storage cp -r ./data/ gs://my-bucket/data/

# Upload with parallel threads
gcloud storage cp -r --parallel-composite-upload-threshold=150M ./data/ gs://my-bucket/

# Upload to BigQuery from local file
bq load --source_format=CSV \
  --autodetect \
  PROJECT_ID:dataset.table \
  ./data.csv

# Upload to BigQuery from Cloud Storage
bq load --source_format=CSV \
  --autodetect \
  PROJECT_ID:dataset.table \
  gs://my-bucket/data/*.csv
```

Loading Data from Cloud Storage

```
# Load CSV into BigQuery
bq load \
  --source_format=CSV \
  --skip_leading_rows=1 \
  --autodetect \
  PROJECT_ID:dataset.table \
  gs://my-bucket/data/*.csv

# Load JSON into BigQuery
bq load \
  --source_format=NEWLINE_DELIMITED_JSON \
  --autodetect \
  PROJECT_ID:dataset.table \
  gs://my-bucket/data/*.json

# Load Avro into BigQuery (schema auto-detected)
bq load \
  --source_format=AVRO \
  PROJECT_ID:dataset.table \
  gs://my-bucket/data/*.avro

# Load Parquet into BigQuery
bq load \
  --source_format=PARQUET \
  PROJECT_ID:dataset.table \
  gs://my-bucket/data/*.parquet

# Import data into Cloud SQL
gcloud sql import csv my-instance \
  gs://my-bucket/data.csv \
  --database=my-database \
  --table=my-table

# Import SQL dump into Cloud SQL
gcloud sql import sql my-instance \
  gs://my-bucket/dump.sql \
  --database=my-database
```

Storage Transfer Service

For large-scale data transfers:

```
# Transfer between Cloud Storage buckets
gcloud transfer jobs create \
  gs://source-bucket \
  gs://destination-bucket \
  --name=my-transfer-job

# Transfer from AWS S3
gcloud transfer jobs create \
  s3://source-bucket \
  gs://destination-bucket \
  --source-creds-file=s3-creds.json

# Transfer from HTTP/HTTPS URL list
gcloud transfer jobs create \
  https://example.com/file-list.tsv \
  gs://destination-bucket

# Schedule a recurring transfer
gcloud transfer jobs create \
  gs://source-bucket \
  gs://destination-bucket \
  --schedule-starts=2025-01-01 \
  --schedule-repeats-every=1d
```

Transfer Options Comparison

Method	Best For	Scale
<code>gcloud storage cp</code>	Small/medium files, ad-hoc	GBs
<code>bq load</code>	Loading into BigQuery	GBs-TBs
Storage Transfer Service	Large-scale, scheduled, cross-cloud	TBs-PBs
Transfer Appliance	Offline transfer (physical device)	PBs
Dataflow	Transform-and-load pipelines	TBs

Exam Practice Questions

1. You need to set up a MySQL database with automatic failover for a production web application. How should you configure Cloud SQL?
- o Answer: Create a Cloud SQL MySQL instance with `--availability-type=REGIONAL`. This creates a failover replica in a different zone with automatic failover.

2. **You need to load 500 TB of data from an on-premises data center to Cloud Storage. Network transfer would take months. What should you use?**

- Answer: **Transfer Appliance** — A physical device shipped to your location, loaded with data, and shipped back to Google for upload.

3. **A data team needs to import CSV files from Cloud Storage into BigQuery daily. What's the simplest approach?**

- Answer: Use `bq load --source_format=CSV --autodetect` to load from Cloud Storage. For daily automation, use **Cloud Scheduler + Cloud Functions** or a **Dataflow template**.

4. **You're deploying Firestore for a new mobile app. You chose Datastore mode but later realize you need real-time listeners. What should you do?**

- Answer: You **cannot change the mode** after creation. You need to create a **new Firestore database in Native mode** and migrate the data.

5. **What is the difference between a Pub/Sub pull and push subscription?**

- Answer: **Pull**: The subscriber explicitly requests messages from the service. **Push**: The service automatically delivers messages to an HTTP endpoint. Use push for Cloud Run/Cloud Functions; use pull for custom processing applications.

Glossary

Acknowledgement (Pub/Sub) — A signal sent by a subscriber to Pub/Sub confirming that a message has been successfully processed. Unacknowledged messages are redelivered until the ack deadline expires or message retention expires.

AlloyDB — A fully managed, PostgreSQL-compatible database service from Google Cloud, designed for high-performance OLTP workloads. Provides PostgreSQL compatibility with significantly higher throughput than standard Cloud SQL PostgreSQL.

Apache Beam — An open-source, unified programming model for defining both batch and streaming data-processing pipelines. Google Dataflow is a fully managed runner for Apache Beam.

Avro — A compact, binary row-based data serialization format used as a file format for BigQuery loads and Dataflow pipelines. Schema is embedded in the file.

BigQuery — GCP's fully managed, serverless, petabyte-scale data warehouse optimized for analytical queries using SQL. Supports columnar storage, partitioning, clustering, and federated queries via external tables.

BigQuery Subscription — A Pub/Sub subscription type that writes incoming messages directly to a BigQuery table, without requiring a consumer application.

Bigtable (Cloud Bigtable) — GCP's fully managed, wide-column NoSQL database with an HBase-compatible API, designed for petabytes of structured data with single-digit millisecond latency. Ideal for time-series, IoT, and analytics workloads.

bin-log (Binary Log) — A MySQL log that records all data modifications. Required on Cloud SQL MySQL instances for point-in-time recovery (PITR) (`--enable-bin-log`).

bq — The command-line tool for interacting with BigQuery. Used to create datasets and tables, load data, run queries, and manage BigQuery resources.

Bucket — The fundamental container for storing objects in Cloud Storage. Buckets have a globally unique name and belong to a specific GCP project.

cbt — The command-line tool for interacting with Cloud Bigtable. Used to create tables, manage column families, and read/write data rows.

Cloud Functions — GCP's serverless Function as a Service offering. Can be triggered by Pub/Sub messages or Cloud Storage events for data processing pipelines.

Cloud Run — GCP's fully managed serverless container platform. Used in data pipelines as a consumer of Pub/Sub push subscriptions and as an Eventarc trigger target.

Cloud Scheduler — GCP's fully managed cron job service. Used to schedule periodic BigQuery loads, Dataflow jobs, or Cloud Functions invocations for batch data pipelines.

Cloud SQL — GCP's fully managed relational database service supporting MySQL, PostgreSQL, and SQL Server. Provides automated backups, high availability (regional failover replicas), and Cloud SQL Auth Proxy for secure connectivity.

Cloud SQL Auth Proxy — A client-side proxy that provides secure, IAM-authenticated connectivity to Cloud SQL instances without needing to allowlist IP addresses or manage SSL certificates manually.

Cloud Storage — GCP's globally unified, scalable object storage service. Stores data in buckets as objects. Used as a staging area for BigQuery loads, Dataflow pipelines, and database backups.

Cloud Storage Subscription — A Pub/Sub subscription type that writes incoming messages to files in a Cloud Storage bucket with configurable prefix and suffix.

Clustered Table (BigQuery) — A BigQuery table in which data is sorted within partitions by one or more specified columns, reducing the amount of data scanned for queries filtering on those columns.

Column Family — A Bigtable concept grouping related columns together. Each column family has its own garbage collection (GC) policy for managing data retention.

Composite Index (Firestore) — A manually created Firestore index over multiple fields, required for queries that filter or sort on more than one field simultaneously.

CSV (Comma-Separated Values) — A plain-text tabular data format supported by BigQuery loads (`bq load --source_format=CSV`) and Cloud SQL imports.

Dataflow — GCP's fully managed service for stream and batch data processing pipelines, based on the Apache Beam programming model. Supports autoscaling workers and exactly-once processing semantics.

Dataflow Template — A pre-packaged, reusable Dataflow pipeline configuration provided by Google for common data movement patterns (e.g., Cloud Storage to BigQuery, Pub/Sub to BigQuery).

Datastore Mode (Firestore) — A Firestore operational mode compatible with the legacy Cloud Datastore API. Suitable for server-side applications; does not support real-time listeners or subcollections in the same way as Native mode.

DDL (Data Definition Language) — SQL statements used to define and modify database schemas (e.g., `CREATE TABLE` , `ALTER TABLE`). Used with Cloud Spanner via `gcloud spanner databases ddl update` .

Dead Letter Topic — A Pub/Sub topic configured to receive messages that failed delivery after a maximum number of retry attempts, enabling analysis and reprocessing of failed messages.

Dual-region — A Cloud Spanner instance configuration that replicates data across two specific geographic regions, balancing availability and latency.

Entity Group (Datastore) — A Datastore-mode concept grouping related entities for strongly consistent reads and transactional writes.

ETL (Extract, Transform, Load) — A data integration process that extracts data from sources, transforms it, and loads it into a target system (e.g., BigQuery). Dataflow is GCP's primary ETL service.

Eventual Consistency — A consistency model in which replicated data will become consistent over time but may temporarily be out of sync. Bigtable multi-cluster replication uses eventual consistency.

External Table (BigQuery) — A BigQuery table whose schema is stored in BigQuery but whose data resides externally (e.g., in Cloud Storage, Bigtable, or Google Drive). Allows querying without loading data.

Failover Replica (Cloud SQL) — A standby Cloud SQL instance in a different zone that automatically takes over if the primary instance fails. Created when `--availability-type=REGIONAL` is set.

Firestore — GCP's fully managed, serverless NoSQL document database that supports real-time synchronization and offline support. Operates in either Native mode or Datastore mode.

GCP (Google Cloud Platform) — Google's suite of cloud computing services, including compute, storage, networking, databases, analytics, and machine learning.

gcloud — The primary command-line tool for interacting with GCP services, part of the Google Cloud SDK.

gcloud storage — The modern GCP command-line tool for Cloud Storage operations, replacing the older `gsutil` tool. Supports file uploads, bucket management, and lifecycle configuration.

GCS (Google Cloud Storage) — An alternative abbreviation for Cloud Storage. Commonly seen in Dataflow pipeline parameters (e.g., `--gcs-location`).

Go — A statically typed, compiled programming language developed by Google. Supported as a Dataflow and Cloud Functions SDK language.

gsutil — The legacy command-line tool for Cloud Storage operations, now largely superseded by `gcloud storage`. Still encountered in documentation and older scripts.

HA (High Availability) — A system design approach that eliminates single points of failure. Cloud SQL achieves HA with `--availability-type=REGIONAL`; Bigtable achieves HA through multi-cluster replication; Redis in Memorystore achieves HA with the STANDARD_HA tier.

HBase — An open-source, wide-column NoSQL database. Cloud Bigtable provides an HBase-compatible API, enabling migration of existing HBase applications.

Hotspotting — A Bigtable performance problem caused by row keys that concentrate read/write traffic on a single node (e.g., sequential timestamps as row key prefixes). Avoided by using reversed timestamps or hash prefixes.

IAM (Identity and Access Management) — GCP's system for controlling access to resources. Used to authorize Cloud SQL Auth Proxy connections, BigQuery dataset access, Pub/Sub publisher/subscriber roles, and more.

IoT (Internet of Things) — Networks of physical devices generating continuous streams of telemetry data. Bigtable is commonly used as a time-series store for IoT sensor data.

Java — A widely used, object-oriented programming language. Supported as an Apache Beam / Dataflow SDK language.

JSON (JavaScript Object Notation) — A lightweight, human-readable data interchange format. Supported as a BigQuery load format (`NEWLINE_DELIMITED_JSON`) and used for Firestore document data.

Lifecycle Rule (Cloud Storage) — A policy applied to a Cloud Storage bucket that automatically transitions objects to a different storage class or deletes them based on age or other conditions.

Materialized View (BigQuery) — A BigQuery table that stores precomputed query results and is automatically refreshed when the underlying base table changes, improving query performance for repeated queries.

Memcached — An in-memory, distributed caching system. Google Cloud Memorystore offers a fully managed Memcached service. Stateless (no persistence), supports multi-threading.

Memorystore — GCP's fully managed in-memory data store service, offering fully managed Redis and Memcached instances within a VPC network (private IP access only).

Message Retention — A Pub/Sub subscription setting defining how long unacknowledged messages are retained (default 7 days, maximum 7 days). Messages are automatically deleted after this period.

Multi-region — A GCP storage or database configuration that replicates data across multiple geographic regions. Cloud Spanner multi-region configurations and Cloud Storage multi-region buckets use this pattern.

MySQL — An open-source relational database management system. Supported by Cloud SQL with versions 5.7 and 8.0. Requires `--enable-bin-log` for point-in-time recovery.

Native Mode (Firestore) — The default and recommended Firestore operational mode supporting real-time listeners, offline client SDKs, and hierarchical subcollections. Best for mobile and web applications.

NEARLINE — A Cloud Storage storage class designed for data accessed less than once per month, offering lower storage costs than STANDARD with a minimum storage duration of 30 days.

NoSQL — A category of database systems that do not use the traditional relational (SQL) model. GCP NoSQL offerings include Firestore, Bigtable, and Memorystore.

Ordering Key (Pub/Sub) — A Pub/Sub message attribute that guarantees messages with the same key are delivered to subscribers in the order they were published.

Parquet — A column-oriented binary data storage format optimized for analytics. Supported as a BigQuery load format (`--source_format=PARQUET`) and commonly used in data lakes.

Partitioned Table (BigQuery) — A BigQuery table divided into segments (partitions) based on a date, timestamp, integer range, or ingestion time column. Reduces query cost by scanning only relevant partitions.

PITR (Point-in-Time Recovery) — The ability to restore a database to any specific point in time using transaction logs. Enabled in Cloud SQL MySQL with `--enable-bin-log`.

PostgreSQL — An open-source, standards-compliant relational database. Supported by Cloud SQL (versions 12–15) and the underlying engine for AlloyDB.

Processing Units — The unit of compute capacity for Cloud Spanner instances. 1 node = 1,000 processing units. Can be scaled in increments of 100 for finer granularity.

Pub/Sub (Google Cloud Pub/Sub) — GCP's fully managed, real-time messaging service that decouples producers and consumers. Supports pull and push delivery models, message ordering, filtering, and dead-letter topics.

Python — A general-purpose programming language. Supported as an Apache Beam / Dataflow SDK language, Cloud Functions runtime, and used in BigQuery client libraries.

REGIONAL (Cloud SQL availability type) — A Cloud SQL high-availability configuration that maintains a synchronous standby instance in a different zone within the same region, providing automatic failover.

Redis — An in-memory data structure store used as a cache, session store, and message broker. Google Cloud Memorystore provides a fully managed Redis service with BASIC and STANDARD_HA tiers.

Retention Policy (Cloud Storage) — A bucket-level lock that prevents objects from being deleted or replaced before a specified minimum retention period expires. Used for compliance (WORM storage).

Row Key — The primary identifier for a row in Bigtable. It is the only index, and all reads are performed by row key or row key range. Row key design is critical for query performance and load distribution.

S3 (Amazon Simple Storage Service) — AWS's object storage service. Storage Transfer Service can transfer data from S3 to Cloud Storage.

Single-field Index (Firestore) — An index automatically created by Firestore for each field in a document, enabling queries on that field without manual index creation.

Spanner (Cloud Spanner) — GCP's fully managed, horizontally scalable, globally distributed relational database that provides external consistency (strong consistency) across regions with 99.999% SLA for multi-region configurations.

SQL Server — Microsoft's enterprise relational database management system. Supported by Cloud SQL in versions 2017, 2019, and 2022.

SSD (Storage Type) — Solid-state drive storage option for Cloud SQL instances, offering higher IOPS compared to HDD. Specified with `--storage-type=SSD`.

STANDARD (Cloud Storage class) — The default Cloud Storage storage class for frequently accessed data, offering the lowest latency and highest throughput with no minimum storage duration.

STANDARD_HA (Memorystore Redis tier) — A Memorystore Redis tier that provides a primary instance with a synchronous replica and automatic failover, suitable for production workloads requiring high availability.

Storage Transfer Service — A GCP managed service for large-scale, scheduled data transfers between Cloud Storage buckets, from AWS S3, or from HTTP/HTTPS URL lists. Suitable for terabyte-to-petabyte transfers.

Subcollection (Firestore) — A collection nested within a Firestore document, allowing hierarchical data modeling. Available only in Native mode.

Subscription (Pub/Sub) — A named resource representing a subscriber's interest in messages from a specific Pub/Sub topic. Subscriptions can be pull-based, push-based, BigQuery-based, or

Cloud Storage-based.

Topic (Pub/Sub) — A named channel to which publishers send messages. Each topic can have multiple subscriptions, with each subscription receiving an independent copy of every message.

Transfer Appliance — A physical, high-capacity storage device shipped by Google to a customer's data center for offline bulk data ingestion. Used when network transfer of petabytes would be impractical.

Uniform Bucket-Level Access — A Cloud Storage access control mode that disables object-level ACLs and enforces access exclusively through IAM policies at the bucket level, simplifying permission management.

VPC (Virtual Private Cloud) — GCP's global, software-defined private network. Memorystore instances are accessible only via private IP within the same VPC; Cloud SQL can be configured for private IP connectivity within a VPC.

Versioning (Cloud Storage) — A Cloud Storage bucket feature that retains older versions of objects when they are overwritten or deleted, enabling recovery of previous object states.

View (BigQuery) — A virtual BigQuery table defined by a SQL query. Queries against a view execute the underlying SQL at query time; no data is stored separately.

WORM (Write Once Read Many) — A data storage model in which data cannot be modified or deleted after writing. Cloud Storage retention policies implement WORM compliance.

ZONAL (Cloud SQL availability type) — A Cloud SQL configuration that places the database instance in a single zone with no standby replica. Lower cost but with no automatic failover.